목차

- Windows 앱 만들기
  - 데스크톱 앱 만들기
    - 개요
      - 앱 플랫폼 선택하기
      - 준비하기
        - 장치에서 개발할 수 있도록 하기
        - 개발자 모드 기능과 디버깅
        - 개발자 계정 생성
    - Windows 앱을 위한 Visual Studio 템플릿
    - 데스크 톱 앱 현대화하기
      - 개요
      - Windows 런타임 API 호출
        - 개요
        - Windows 런타임 API 사용 가능하게
      - WinRT XAML controls 호스팅(XAML Islands)
        - 개요
        - XWPF 와 Windows Forms apps 에서 XAML Islands
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
        - C++ Win32 앱에서 XAML Islands
          - WinRT XAML hosting API 사용
          - 표준 WinRT XAML control 호스트
          - 커스텀 WinRT XAML control 호스트
          - 응용 시나리오
      - Visual layer (Windows.UI.Composition) 사용하기
        - 개요
        - WPF
        - Windows Forms
        - Win32
      - 패키지 ID 가 필요한 기능
        - 개요
        - 패키지 확장자로 통합
        - 파일 탐색기로 패키지된 데스크톱 앱 통합

            - 패키지된 앱에 대한 활성화 정보 가져오기
            - UWP 구성 요소로 확장
            - 패키지된 앱 배포
        - 튜토리얼: WPF 앱 현대화하기
            - 개요
            - Contoso Expenses 앱을 .NET Core 3 로 마이그레이션
            - XAML Islands 를 이용하여 UWP 잉크캔버스 컨트롤 추가
            - XAML Islands 를 이용하여 UWP 캘린더뷰 컨트롤 추가
            - Windows 10 사용자 활동과 알림 추가
            - MSIX 로 패키지/배포
        - 패키징되지 않은 데스크톱 앱에 ID 부여
    - Windows UI 라이브러리(WinUI)
        - 개요
        - Windows UI Library 3 Preview 4
            - 개요 및 릴리스 노트
            - WinUI 3 으로 데스크톱 앱 만들기 시작하기
            - WinUI 3 으로 UWP 앱 만들기 시작하기
            - C++/WinRT 를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
            - C#를 사용하는 WinUI 3 앱에 대한 XAML 컨트롤 템플릿
        - Windows UI 2.x 라이브러리
            - 개요
            - 셋업
                - WinUI 2.x 시작해 보기
                - NuGet 패키지 목록
            - 릴리스 노트
                - 개요
                - WinUI 2.5
                - WinUI 2.4
                - WinUI 2.3
                - WinUI 2.2
                - WinUI 2.1
                - WinUI 2.0
        - WinUI API 레퍼런스
    - Project Reunion 으로 Windows 앱 제작(사전 릴리스)

- 개요
- MRT Core
- DWriteCore

- Windows 10X 개발
  - Windows 10X 개발 FAQ
- Fluent Design System
- 접근성
- 스피치
- 기능 및 기술
- 최근 문서 업데이트
- Microsoft Build 2020 관련 문서

Windows PC 데스크톱 앱 개발하기

2/18/2021

이 자료에서는 Windows 용 데스크톱 앱을 만들거나 Windows 10 의 최신 환경을 적용하기 위해 기존 데스크톱 앱을 업데이트하는 데 필요한 정보를 제공합니다.

데스크톱 앱 플랫폼

Windows PC 용 데스크톱 앱을 구축하는 데는 네 가지 주요 플랫폼이 있습니다. 각 플랫폼은 앱의 라이프사이클을 정의하는 앱 모델, Word, Excel, Photoshop 과 같은 데스크톱 앱을 생성하고 Windows 기능을 사용하기 위해 관리되는 API 또는 기본 API 에 액세스할 수 있는 UI 프레임워크 및 UI 컨트롤 세트를 제공합니다.

추가적인 자료와 함께 세세한 비교를 보려면, 앱 플랫폼 선택하기 부분을 보십시오.

플랫폼	설명	문서 등 자료
Universal Windows Platform (UWP)	Windows 10 앱 및 게임을 위한 최첨단 플랫폼. UWP 컨트롤과 API 를 독점적으로 사용하는 UWP 앱을 구축하거나, 다른 플랫폼 중 하나를 사용하여 구축된 데스크톱 앱에서 UWP 컨트롤과 API 를 사용할 수 있습니다.	시작하기 API 레퍼런스 샘플
C++/Win32	Windows 및 하드웨어에 직접 액세스해야 하는 기본 Windows 앱의 플랫폼입니다.	시작하기 API 레퍼런스 샘플
WPF	XAML UI 모델을 사용하여 그래픽 기반으로 관리되는 Windows 애플리케이션을 위한 확립된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET) 샘플
Windows Forms	경량 UI 모델로 관리되는 LOB(Line-of-Business) 애플리케이션용으로 설계된 .NET 기반 플랫폼. .NET Core 3 또는 모든 .NET Framework 를 대상으로 할 수 있습니다.	시작하기 API 레퍼런스(.NET)

향후 로드맵

앞으로 우리는 Windows UI Library(WinUI) 및 Project Reunion 을 통해 Windows 앱 개발 플랫폼을 발전시키고 있습니다.

- **WinUI** 는 Windows 10 앱용 기본 UX(사용자 환경) 프레임워크입니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 WinUI 3(현재 프리뷰)인 WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 추가적인 정보는 Windows UI 라이브러리 (WinUI) 자료를 보십시오.
- **Project Reunion**(현재 프리뷰)은 Windows 앱 개발 플랫폼의 다음 진화를 나타내는 광범위한 새로운 개발자 구성 요소 및 도구의 태명입니다. Project Reunion 은 광범위한 대상 윈도우즈 10 OS 버전의 모든 앱에서 일관된 방식으로 사용할 수 있는 통합 API 및 툴 세트를 제공합니다. Project Reunion 은 UWP 및 네이티브 Win32, dotNET 과 같은 기존 Windows 앱 플랫폼과 프레임워크를 개발자들이 이러한 플랫폼 전반에 걸쳐 의존할 수 있는 공통 API 및 툴셋으로 보완합니다. 추가적인 정보는 Project Reunion 자료를 보십시오.

기존 데스크톱 앱을 Windows 10 용으로 업데이트

기존 WPF, Windows Forms 또는 기본 Win32 데스크톱 앱이 있는 경우 Windows 10 및 UWP(Universal Windows Platform)는 앱에서 최신 환경을 제공하는 데 사용할 수 있는 많은 기능을 제공합니다. 이러한 기능 중 대부분은 다른 플랫폼을 위해 앱을 다시 작성할 필요 없이 자신만의 속도로 앱에서 채택할 수 있는 모듈형 구성 요소로 제공됩니다.

기존 데스크톱 앱을 개선하는 데 사용할 수 있는 몇 가지 기능:

- MSIX 를 사용하여 데스크톱 앱을 패키징 및 배포합니다. MSIX 는 모든 Windows 앱에 범용 패키지 환경을 제공하는 최신 Windows 앱 패키지 형식입니다. MSIX 는 MSI, .appx, App-V 및 ClickOnce 설치 기술의 가장 좋은 측면을 결합하며, 안전하고 신뢰할 수 있도록 구축되었습니다.
- **패키지 확장**을 사용하여 데스크톱 앱을 Windows 10 과 통합할 수 있습니다. 예를 들어, 앱의 타일을 시작하거나, 앱을 공유 대상으로 만들거나, 앱에서 토스트 알림을 보냅니다.

  XAML Islands 를 사용하여 UWP XAML 컨트롤을 데스크톱 앱에서 호스팅합니다. 대부분의 최신 Windows 10 UI 기능은 UWP XAML 컨트롤에서만 사용할 수 있습니다.

추가적인 정보는 다음 자료를 보십시오.

자료	설명
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
튜토리얼: WPF 앱 현대화하기	UWP 잉크 및 캘린더 컨트롤을 앱에 추가하고 MSIX 패키지에 패키징하여 기존 WPF LOB(Line-of-Business) 샘플 앱을 현대화하려면 단계별 지침을 따르십시오.

새 데스크톱 앱 만들기

새 Windows 앱을 만들고자 한다면 다음 자료가 시작하는 데 도움이 될 수 있습니다.

자료	설명
앱 플랫폼 선택하기	주요 데스크톱 앱 플랫폼을 심층적으로 비교하고 사용자의 요구에 적합한 플랫폼을 선택하는 데 도움이 됩니다. 이 문서에서는 각 플랫폼에 대한 문서에 대한 유용한 링크도 제공합니다.
Windows 앱을 위한 Visual Studio 프로젝트 템플릿	Visual Studio 에서 C# 또는 C++를 사용하여 Windows 10 장치용 앱을 만드는 데 도움이 되는 프로젝트 및 항목 템플릿을 설명합니다.
데스크톱 앱 현대화하기	WPF, Windows Forms 및 C++ Win32 앱을 포함한 모든 데스크톱 앱에서 사용할 수 있는 최신 Windows 10 및 UWP 개발 기능에 대해 설명합니다.
기능 및 기술	각 기본 데스크톱 앱 플랫폼과 관련 문서에 대한 링크를 통해 액세스할 수 있는 Windows 기능에 대한 개요를 제공합니다.

관련 문서 및 기술

자료	설명
.NET Core 3.1	WPF/Windows Forms 앱을 위한 개선을 비롯한 .NET Core 3.1 의 최신 기능에 대하여 설명합니다.
.NET 5	이 자료에는 3.1 의 .NET Core 다음 릴리스인 .NET 5 에 포함된 내용이 자세히 나와 있습니다.
Desktop guide for WPF and .NET Core	모든 .NET 프레임워크가 아닌 .NET Core 대상 WPF 앱 개발
Azure	Azure 클라우드 서비스를 통해 애플리케이션의 범위를 확장하십시오.
Visual Studio	앱 및 서비스를 개발하기 위해 Visual Studio 사용법을 배우십시오.
MSIX	모든 Windows 앱을 최신 범용 패키지 형식으로 패키징 및 배포합니다.
Windows AI	Windows AI 를 사용하여 앱의 복잡한 문제에 대한 지능형 솔루션을 구축합니다.
Windows Containers	완전히 격리된 빠른 윈도우즈 환경에서 애플리케이션을 종속성으로 패키지화하십시오.
Progressive Web Apps	웹 앱을 Windows 10 에서 UWP 앱으로 배포하고 실행할 수 있는 진행형 웹 앱으로 변환합니다.
Xamarin	.NET 코드 및 플랫폼별 사용자 인터페이스를 사용하여 Windows, Android, iOS 및 macOS 용 크로스 플랫폼 앱을 구축합니다.
Docs archive for Windows 8.x and earlier	Windows 8.x 및 이전 버전의 앱 작성에 대한 보관된 문서에 액세스합니다.

Windows 앱 플랫폼 선택하기

2/18/2021

Windows PC 용 새 데스크톱 응용 프로그램을 만들려는 경우 가장 먼저 사용할 응용 프로그램 플랫폼을 결정합니다. Windows(윈도우)는 4 가지 주요 애플리케이션 플랫폼을 제공하며 각각 다른 강점을 가지고 있습니다.

- Universal Windows Platform (UWP): 이 플랫폼은 Windows 10 을 실행하는 모든 장치에 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다. 네이티브 또는 관리 가능 프로그램이 있습니다.
- WPF and Windows Forms: 이러한 .NET 기반 플랫폼은 중앙 관리 애플리케이션을 위한 공통 유형 시스템, API 및 애플리케이션 모델을 제공합니다.
- Win32: 이 플랫폼은 Windows 및 하드웨어에 직접 액세스해야 하는 기본 C/C++ Windows 애플리케이션을 위한 원래 플랫폼입니다. 따라서 Win32 API 는 최고 수준의 성능과 시스템 하드웨어에 직접 액세스해야 하는 애플리케이션을 위한 선택 플랫폼으로 자리 잡게 됩니다.

각 플랫폼에는 완벽한 UI 프레임워크와 Word, Excel 및 Photoshop 과 같은 데스크톱 앱을 생성하여 기존 Windows 데스크톱에서 실행되며 해당 환경의 특정 기능을 최대한 활용할 수 있는 UI 컨트롤 세트가 포함되어 있습니다. Windows 10 에서는 각 플랫폼에서도 Windows UI 라이브러리를 사용하여 사용자 인터페이스를 만들 수 있습니다.

이러한 플랫폼 중 일부는 일부 특성을 공유하며 특정 유형의 애플리케이션에 더 적합합니다. 예를 들어 UWP 와 .NET 은 Visual Studio 와 긴밀하게 통합됩니다. 이는 특히 개발자 생산성, 정교하고 사용자 정의가 가능한 UI 및 애플리케이션 보안 분야에서 많은 이점을 제공합니다. 이러한 프레임워크는 UI 를 빠르게 생성하는 시각 디자이너와 UI 마크업을 지원하기 때문에 특히 LOB(Line-of-Business) 애플리케이션에 적합합니다.

> **NOTE**
> 어떤 앱 플랫폼을 선택하든 많은 Windows 10 기능을 사용하여 앱에서 현대적인 경험을 제공할 수 있습니다. 예를 들어 WPF, Windows Forms 또는 Win32 API 를 사용하여 데스크톱 앱을 빌드한 경우에도 MSIX 패키지 배포를 계속 사용할 수 있습니다. 데스크톱 앱을 현대화하는 모든 방법에 대한 자세한 내용은 데스크톱 앱 현대화를 참조하십시오.

UWP

UWP 는 Windows 10 응용 프로그램과 게임을 위한 최첨단 플랫폼입니다. XAML 마크업을 사용하여 UI 와 코드(비즈니스 논리)를 구분하는 사용자 지정이 가능한 플랫폼입니다. UWP 는 정교한 UI, 스타일 사용자 지정 및 그래픽 집약적인 시나리오가 필요한 데스크톱 애플리케이션에 적합합니다. 또한 UWP 는 기본 UX 환경을 위한 Fluent Design System 을 기본적으로 지원하며 Windows Runtime(WinRT) API 에 대한 액세스를 제공합니다. Fluent 를 채택함으로써 UWP 는 잉크, 터치, 게임패드, 키보드, 마우스와 같은 일반적인 입력 방식을 자동으로 지원합니다.

UWP 를 사용하여 Windows PC 용 데스크톱 응용 프로그램을 만들 수 있을 뿐만 아니라 Xbox, HoloLens 및 Surface Hub 응용 프로그램에서 유일하게 지원되는 플랫폼입니다. UWP 는 최신 첨단 애플리케이션 플랫폼입니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기
- 프로젝트 템플릿
- 디자인과 UI
- 기술 및 기능
- API 레퍼런스
- 샘플

WPF

WPF 는 .NET Core 또는 full .NET Framework 에 액세스할 수 있는 관리되는 Windows 애플리케이션을 위한 플랫폼이며 XAML 마크업을 사용하여 UI 와 코드를 구분한다. 이 플랫폼은 정교한 UI, 스타일 사용자 지정 및 그래픽

집약적인 시나리오가 필요한 데스크톱 애플리케이션을 위해 설계되었습니다. WPF 개발 기술은 UWP 개발 기술과 유사하므로 WPF 에서 UWP 앱으로 마이그레이션하는 것이 Windows Forms 에서 마이그레이션하는 것보다 쉽습니다.

추가적인 정보는 다음 자료들에 있습니다.

- 시작하기(WPF)
- 프로젝트 템플릿
- 첫 앱 만들기 (.NET Core)
- 첫 앱 만들기 (.NET Framework)
- WPF 앱을 .NET Core 로 마이그레이션
- API 레퍼런스(.NET)
- 샘플

Windows Forms

Windows Forms 는 .NET Core 또는 full .NET 프레임워크에 대한 액세스와 가벼운 UI 모델이 있는 관리되는 Windows 응용 프로그램을 위한 원래 플랫폼입니다. 개발자가 플랫폼을 처음 사용하는 개발자에게도 애플리케이션 구축을 신속하게 시작할 수 있도록 지원하는 데 탁월합니다. 이 플랫폼은 시각적 및 비시각적 드래그 앤 드롭 제어 기능이 내장된 폼 기반 신속한 애플리케이션 개발 플랫폼입니다. Windows Forms 는 XAML 을 사용하지 않으므로 나중에 응용 프로그램을 UWP 로 확장하기로 결정하려면 UI 를 완전히 다시 써야 합니다.

추가적인 정보는 다음 자료들에 있습니다.

- Windows Forms 시작하기
- 프로젝트 템플릿
- 첫 Windows Forms 앱 만들기
- 튜토리얼: 사진 뷰어 만들기
- API 레퍼런스 (.NET)
- Windows Forms 앱 개선하기

Win32

Win32 API 를 C++와 함께 사용하면 WinRT 와 .NET 과 같은 관리 런타임 환경에서 가능한 것보다 관리되지 않은 코드로 대상 플랫폼을 더 많이 제어함으로써 최고 수준의 성능과 효율성을 달성할 수 있습니다. 그러나 애플리케이션 실행을 이러한 수준으로 제어하려면 올바르게 하려면 더 많은 주의를 기울여야 하며, 개발 생산성이 대체로 낮습니다.

다음은 고성능 애플리케이션을 구축할 수 있도록 Win32 API 및 C++가 제공하는 기능에 대한 몇 가지 강조 사항입니다.

- 리소스 할당, 개체 수명, 데이터 레이아웃, 정렬, 바이트 패킹 등에 대한 엄격한 제어를 포함한 하드웨어 수준 최적화
- 내장 기능을 통해 SSE 및 AVX 와 같은 성능 지향 명령 집합에 액세스
- 템플릿을 이용한 효율적이고 안전한 제네릭 프로그래밍
- 효율적이고 안전한 컨테이너/알고리즘
- DirectX, 특히 Direct3D 및 DirectCompute(UWP 는 DirectX interop 도 제공.)

추가적인 정보는 다음 자료들에 있습니다.For more information, see the following articles:

- 시작하기
- 프로젝트 템플릿
- 첫 Win32/C++ 앱 만들기
- 기술 및 기능
- API 레퍼런스
- 샘플

플랫폼 비교: UWP, WPF, and Windows Forms

다음 표는 위 플랫폼에 대한 더 자세한 비교를 보여줍니다.

기능/시나리오	UWP	WPF	WINDOWS FORMS
지원 버전	Windows 10	Windows 7 이후	Windows 7 이후
언어	C#, C++/WinRT, C++/CX, VB, JavaScript	C#, C++/CLI (C++ 확장), F#, VB	C#, C++/CLI (C++ 확장), F#, VB
UI 런타임	Native (C++/WinRT and C++/CX) and managed (.NET Native)	Managed (.NET Framework and .NET Core 3)	Managed (.NET Framework and .NET Core 3)
오픈 소스	Yes (Windows UI Library only)	Yes (.NET Core only)	Yes (.NET Core only)
XAML 지원	Yes	Yes	No
강점	- XAML markup for UI - 풍부한 커스텀 UX - 기존 코드 기반이 .NET 표준과 호환 - 고 DPI 지원 - Windows 장치 간 다중 입력 유형 지원(터치, 펜, 게임패드, 마우스, 키보드 등) - Xbox, HoloLens, IoT or Surface Hub 지원 - 선택적 밀집 UI - 네이티브 C++ 지원 - 배터리 수명 최적화 - 최신 접근성 지원(읽어주기 등) - 많은 유용한 텍스트 데이터 (빌트인 스펠체크 등) - 잉크 그림 지원 - 애플리케이션 컨테이너를 통한 안전한 실행(ex: 신뢰할 수 없는 컨텐츠는 안전모드)	- XAML markup for UI - 풍부한 커스텀 UX - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 입력 확인을 위한 플랫폼 지원	- 빠른 개발 - UI 개발을 위한 WYSIWYG 에디터 - 마이크로소프트 및 파트너의 대규모 제어 기능 모음 - 밀집 UI - Windows 7 지원 - 키보드/마우스 입력

지원이 제한된 시나리오	- 다중 창 지원 [1] - 입력 확인을 위한 플랫폼 지원 [1] - Windows 7 미지원 - 몇몇 Windows 런타임 API 가 Windows 10 의 특정 버전만 지원함 - 플랫폼 완전 지원과 셸 통합(예를 들어 UWP 는 현재 시스템 트레이 통합 또는 모든 장치에 대한 전체 액세스를 지원하지 않음) - 디스크의 모든 파일 직접 접근 - ADO.NET - 비.NET 표준 또는 비 Windows App 인증서 Kit 호환 API 를 사용하는 기존 코드베이스 클래스 라이브러리 - 지역 네트워크 루프백 지원(즉, 대상 디바이스에 루프백 예외를 만들지 않고 앱이 localhost 와 통신해야 하는 경우) - 강력한 파일 입출력	- 고 DPI 지원 [2] - 터치 입력 [2]	- 그래픽/UX(터치/애니메이션 등) - 고 DPI 지원 [2] - 터치 입력 [2] - 뷰/데이터 모델의 추상화 수 - 커스텀 가능 UI

[1] 향후 Windows 10 릴리스에서 이 시나리오를 다룰 기능을 공개적으로 발표했습니다.

[2] 플랫폼에는 이 시나리오에 대한 1 급 API 지원이 부족하지만 개발자는 해결 방법을 통해 이 시나리오를 지원할 수 있습니다.

Windows UI 라이브러리를 Windows 앱에 사용하기

기본 Windows 앱 플랫폼을 보완하기 위해 앱에서 Windows UI 라이브러리를 사용할 수도 있습니다. WinUI 는 Windows 10 의 하위 버전을 대상으로 하는 UWP 앱에 대한 WinRT 컨트롤의 새로운 버전과 업데이트된 버전을 제공하는 툴킷으로 시작했습니다. 현재 윈 UI 3(프리뷰) WinUI 는 UWP, .NET 및 Win32 앱 플랫폼 전반에 걸쳐 Windows 10 앱을 위한 UI(기본 사용자 인터페이스) 프레임워크로 성장하고 있습니다. 다음과 같은 방법으로 사용할 수 있습니다.

- WinUI 2.x:
  - UWP 앱은 Windows SDK 에서 제공하는 WinRT 컨트롤 대신 WinUI 2.x 컨트롤을 사용할 수 있습니다. 이 WinUI 릴리스에는 Windows SDK 의 새 컨트롤과 기존 컨트롤의 업데이트된 버전이 모두 포함되어 있습니다.
  - XAML Islands 를 사용하여 WinUI 2.x 컨트롤을 호스팅하도록 기존 WPF, Windows Forms 및 C++/Win32 앱을 업데이트할 수 있습니다.
- WinUI 3 (프리뷰):

WinUI 3 부터는 완전히 WinUI 기반 UI 를 사용하는 dotNET 및 C++/Win32 앱과 UWP 앱을 생성할 수 있습니다. 이 릴리스에는 이러한 앱을 만드는 데 필요한 모든 것을 제공하는 Visual Studio 프로젝트 템플릿이 포함되어 있습니다.

> **NOTE**
> 3 판은 아직 프리뷰이며, 현재 생산성 앱에 사용할 수 없습니다.

다른 앱 플랫폼

Progressive Web Apps (PWAs)

개발자들은 PWA 를 통해 웹 사이트 코드를 패키징하여 Windows 10 PC 에 애플리케이션처럼 설치 및 실행할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Xamarin

Xamarin 을 사용하여 iOS 및 Android 에서도 실행할 수 있는 Windows 10 용 크로스 플랫폼 애플리케이션을 구축할 수 있습니다. 자세한 내용은 여기를 참조하십시오.

Uno Platform

Uno 플랫폼은 Windows UWP 기반 코드(C# 및 XAML)를 iOS, Android, MacOS, Linux 및 Web Assembly 에서 실행할 수 있게 합니다. 윈도우 10 2004(19041)의 UWP 에 대한 전체 API 정의와 UWP API 의 일부를 구현(Windows.UI.Xaml 등)하여 UWP 애플리케이션이 이러한 플랫폼에서 실행될 수 있도록 한다. 자세한 내용은 Uno 플랫폼 문서를 참조하십시오.